



Sistemas Operacionais



getpriority()

Tema 4

Diogo Pontes Vieira Costa
Felipe Almeida Cardoso
Guilherme Silva Cotrim
Matheus Vieira de Assis
Pedro Paulo Veri Marquez

Universidade Federal de Uberlandia



Sumario

1. Tema Escolhido
2. Motivacao do Projeto
3. Algoritmos e Mecanismos Implementados
4. Funcionamento da Solucao
5. Proposta de Implementacao no XV6
6. Encerramento



Tema Escolhido

O tema escolhido pelo grupo consiste na implementacao de um mecanismo de prioridade de processos no sistema operacional XV6, incluindo as chamadas de sistema `getpriority()` e `setpriority()`, alem da integracao com o escalonador do sistema.



Motivacao do Projeto

O XV6 utiliza um escalonador simples baseado em round-robin, onde todos os processos recebem fatias iguais de tempo de CPU. Sistemas operacionais reais, no entanto, utilizam escalonamento baseado em prioridade.

Este projeto visa tornar o XV6 mais realista e educacional, abordando:

- Controle de CPU entre processos
- Prevencao de starvation
- Gerenciamento de processos no kernel



Algoritmos e Mecanismos

1. Armazenamento de Prioridade: Adicao de um campo priority na struct proc do XV6.
2. Chamadas de Sistema: Implementacao de `getpriority(int pid)` e `setpriority(int pid, int prio)`, com tratamento de argumentos, busca de processos e uso de locks.
3. Politica de Escalonamento (proposta): Introducao de aging para prevenir starvation, aumentando gradualmente a prioridade de processos em espera.



Funcionamento da Solucao

1. Processos podem consultar ou alterar prioridades via syscalls.
2. O kernel armazena a informacao de prioridade na estrutura do processo.
3. O escalonador utiliza a prioridade para decidir a ordem de execucao.
4. Processos com maior prioridade sao executados primeiro.

Exemplo: Este comportamento simula o funcionamento de sistemas operacionais reais, onde processos criticos recebem mais tempo de CPU.



Proposta de Implementacao no XV6

Etapa 1: Adicionar campo priority na struct proc.

Etapa 2: Criar syscalls (getpriority, setpriority), registrar em syscall.c e adicionar stubs em user/usys.pl.

Etapa 3: Modificar scheduler() para utilizar prioridade na escolha do proximo processo.

Etapa 4: Validar com testes e analisar o impacto do mecanismo de aging para evitar starvation.



Encerramento

Este projeto estende o XV6 para suportar escalonamento avançado baseado em prioridade, melhorando a compreensão sobre o funcionamento de kernels modernos e sistemas operacionais reais.

